

Selective Consistency Training with Reinforcement Learning

Anonymous Authors¹

Abstract

[TODO: write abstract once results are settled.]

1. Introduction

Large language model (LLM) behaviour is often influenced by prompt cues that should not, in principle, affect the behaviour of interest. A widely studied example is sycophancy, where LLMs adapt their responses to align with a user's stated views or preferences (Perez et al., 2022). Another example is evaluation gaming, where LLM behaviour depends on whether the model "believes" the input to be part of an evaluation. [TODO: cite apollo, anthropic and AISI's research] This is particularly concerning because, if models behave differently in evaluations than in the real world, our evaluations become uninformative about real-world behaviour (Carlsmith, 2023).

Two general approaches address these behavioural inconsistencies. The first is to modify the input or the internal representations induced by the input so that the cues driving the inconsistencies are absent or hard for the LLM to detect [TODO: cite cot & activation steering work on eval awareness reduction]. However, isolating and removing the relevant cues from the input or internal representations is difficult in practice [TODO: why?]. The second is to train LLMs to behave consistently regardless of whether such cues are present. We focus on the second approach in this work.

1.1. Consistency training

Consistency training is a class of methods that train LLMs to produce consistent outputs or internal representations across input perturbations (Chua et al., 2024; Irpan et al., 2025).

Bias-Augmented Consistency Training. Bias-Augmented Consistency Training (BCT) trains LLMs

to produce the same output on perturbed inputs as on the corresponding unperturbed input. For each prompt, a response is sampled from the LLM. The LLM is then fine-tuned to produce that same response when given perturbed prompts (Chua et al., 2024).

Activation Consistency Training. Activation Consistency Training (ACT) trains LLMs to produce internal representations when given a perturbed input that match the corresponding unperturbed input (Irpan et al., 2025). This is achieved by minimising the difference between residual-stream activations on the perturbed and clean prompts at matching token positions, pushing the perturbed activations toward the clean ones. ACT is applied only to prompt token positions, not response token positions. [TODO: mention BCT still SOTA]

Existing applications. Consistency training has previously been applied to sycophancy (Chua et al., 2024), jailbreak robustness (Irpan et al., 2025), and reducing persona drift [TODO: cite @miscshah2026frustration, title = Gemma Gets Help: Mitigating Frustration and Persona Drift with Consistency Training, author = Shah, Neil and Africa, David Demitri, year = 2026, howpublished = LessWrong, note = SPAR Research Fellowship, url =]. Irpan et al. (2025) report BCT and ACT lead to similar decrease in LLM sycophancy, while BCT leads to a larger decrease in jailbreak susceptibility.

1.2. The obfuscation problem and selective consistency training

BCT and ACT train LLMs to generate the same output or internal representations as when certain cues are absent from the prompt. This raises multiple concerns. Irpan et al. (2025) note that this training may misgeneralise to LLMs ignoring other important parts of the prompt, such as user instructions. Furthermore, constraining the entire output or internal activations also constrains any mentions or verbalisations of the prompt cues driving inconsistency, leading to *obfuscation*: the model learns to suppress evidence that it detected the cue. This hampers the applicability of these methods when monitoring and auditing are important. For example, training models to not verbalise prompt injection jailbreaks may reduce their monitorability; training LLMs

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

to not verbalise when they correctly infer that they are being evaluated reduces our ability to track evaluation awareness [TODO: cite eval awareness literature eg. ansti scheming, apollo system cards, igor’s research].

These concerns motivate *selective consistency training*: enforcing consistency only on the behavioural properties of interest, leaving the rest of the output—including any verbalisation of the cues—free to vary. We introduce *Reinforcement Learning Consistency Training* (RLCT), which implements this by computing the rate of the target behaviour under each input perturbation and rewarding outputs that bring those rates into alignment.

1.3. Summary of contributions

- We introduce *Reinforcement Learning Consistency Training* (RLCT), a method that enforces consistency only over selected behavioural properties of the output.
- We validate the method against BCT on the sycophancy benchmark of Chua et al. (2024) on Meta Llama 3.1 8B Instruct and OpenAI GPT OSS 20B.
- We show that RLCT causes significantly less obfuscation compared to BCT, as measured by significantly lower decrease in bias acknowledgment.

2. Reinforcement Learning Consistency Training

2.1. Method

Consider an input x . Let $\mathcal{T}$ denote a set of *training* perturbations and $\mathcal{A}$ a set of *anchor* perturbations (the two sets may overlap), and write $\mathcal{F} = \mathcal{T} \cup \mathcal{A}$ for their union. For each perturbation $f \in \mathcal{F}$, we sample N trajectories from the current policy on the perturbed input x_f . Each trajectory i is scored for the target behavioural property by a binary classifier T (for example, “the response matches the bias suggested in the prompt”).

Define the local behaviour rate for each $f \in \mathcal{F}$ as

$$p_f = \frac{1}{N} \sum_{i=1}^N T(\tau_{f,i}), \quad (1)$$

and let $q = Q(\{p_f\}_{f \in \mathcal{A}})$ denote a target rate aggregated over the anchor perturbations. This target rate may be the behaviour rate on a single anchor perturbation (for example, the clean or unperturbed prompt), or a function of the per-perturbation rates such as their mean, minimum, or maximum.

The consistency reward, applied only to trajectories from training perturbations, rewards a trajectory when its trait value moves the local rate p_f toward the target rate q :

$$r_{f,i}^{\text{cons}} = -(p_f - q)(T(\tau_{f,i}) - p_f), \quad f \in \mathcal{T}. \quad (2)$$

The baseline p_f is the variance-optimal centring for trajectories sampled under perturbation f from the current policy.

To prevent the target rate itself from drifting during training, an optional anchor reward, applied only to trajectories from anchor perturbations, ties q to the same target statistic $q_0 = Q(\{p_f^{(0)}\}_{f \in \mathcal{A}})$ computed under the initial policy π_0 (typically a checkpoint taken before consistency training begins), where $p_f^{(0)}$ is the behaviour rate of π_0 on perturbation f :

$$r_{f,i}^{\text{anchor}} = -(q - q_0)(T(\tau_{f,i}) - q), \quad f \in \mathcal{A}. \quad (3)$$

The baseline q is again the variance-optimal centring under the current sampling distribution: q is the empirical mean of trait values over the anchor rollouts, whereas q_0 comes from a frozen policy and would not yield an unbiased baseline.

The total reward for trajectory $\tau_{f,i}$ combines the two terms with mixing weight α :

$$r_{f,i} = (1 - \alpha)r_{f,i}^{\text{cons}} + \alpha r_{f,i}^{\text{anchor}}, \quad (4)$$

with each component taken as zero outside its domain of definition. A trajectory from a perturbation in $\mathcal{T} \cap \mathcal{A}$ thus receives both components; one from $\mathcal{T} \setminus \mathcal{A}$ receives only the consistency term, and one from $\mathcal{A} \setminus \mathcal{T}$ only the anchor term.

We optimise the policy with GRPO and a KL penalty to a fixed reference policy. The only departure from standard GRPO is the sampling structure: groups consist of trajectories from all $|\mathcal{F}|$ perturbations of a given input, with N trajectories per perturbation; advantage normalisation and policy updates then proceed as usual. We use parameter-efficient fine-tuning (LoRA) throughout.

2.2. Why this avoids the obfuscation pressure

Because the reward depends only on the trait T , the per-perturbation rates p_f , and the chosen target rate q , the model is not penalised for any particular content in the chain of thought. In particular, if the model verbalises that it has noticed a perturbation, the reward is unchanged provided its action matches the target rate. This contrasts with BCT, where matching a clean-prompt target response would penalise verbalisation of the perturbation.

3. Experiments

3.1. Environment and bias types

We validate the method on the sycophancy benchmark of Chua et al. (2024). The benchmark consists of multiple-choice questions to which a small prompt-level perturbation has been added that suggests a particular answer. We use six single-turn bias types from the benchmark: suggested answer, distractor fact, distractor argument, post hoc, spurious

few-shot squares, and wrong few-shot. Brief descriptions are given in Appendix A; see Chua et al. (2024) for full specifications.

3.2. Models

We run experiments on two open-weight models: Meta Llama 3.1 8B Instruct and OpenAI GPT OSS 20B. For Llama, the prompt instructs the model to reason step-by-step before answering. For GPT OSS, no such instruction is added: the model already produces internal reasoning before its final answer in its native response format. All training uses LoRA adapters of rank 8.

3.3. Train/evaluation split

We train on the held-in question sources LogiQA and Hel-laSwag (1024 questions sampled from each, drawn from the test split following the BCT data-generation procedure) and evaluate on a held-out question source, the multiple-choice subset of Humanity’s Last Exam (HLE), with 100 questions per bias type per checkpoint and 400 questions per bias type for the base-model reference. For BCT, an additional 2048 instruction-following samples (prompts drawn from the Cleaned Alpaca dataset, with completions re-sampled from the target model so the supervision is self-distilled) are mixed into the training data to mitigate degradation on unrelated tasks. We train on the distractor-argument bias only and evaluate on all six bias types, which lets us measure transfer of the consistency objective to unseen biases. An additional regime that trains on a mixture of distractor-argument and wrong-few-shot prompts is reported in Appendix E.

3.4. Training settings

BCT is fine-tuned by SFT for one epoch on the 2048 BCT samples plus the 2048 instruction-following samples, at batch size 128 (32 optimisation steps). RLCT is trained with GRPO for one epoch over 64 prompts at batch size 4 prompts per update (16 update steps); each prompt is expanded into a group of $|\mathcal{F}| = 2$ perturbations (biased and unbiased) with $N = 128$ rollouts per perturbation, and the consistency reward uses the unbiased prompt as the target rate q with anchor weight $\alpha = 0$. For each method-model combination we run three learning rates and pool the resulting checkpoints when reporting metrics, which gives a binomial standard error without tripling the rollout budget. Remaining hyperparameters are listed in Appendix B.

3.5. Metrics

For each evaluation question x we sample one response under the biased prompt and one under the unbiased prompt, and let $b(x), u(x) \in \{0, 1\}$ indicate whether the model

selected the biased option on each. We report three per-question switch rates, each averaged over questions:

$$\text{BSR}_{\rightarrow} = \frac{1}{|\mathcal{X}|} \sum_x \max(0, b(x) - u(x)),$$

$$\text{BSR}_{\leftarrow} = \frac{1}{|\mathcal{X}|} \sum_x \max(0, u(x) - b(x)),$$

$$\text{BSR}_{\text{tot}} = \frac{1}{|\mathcal{X}|} \sum_x |b(x) - u(x)|,$$

which we call the *towards-bias switch rate*, the *away-from-bias switch rate*, and the *total switch rate* respectively. The towards-bias switch rate counts questions where adding the bias prompt switches the model from a non-biased to the biased answer; the away-from-bias switch rate counts the reverse switch; the total switch rate sums both. Aggregating per question rather than taking the absolute difference of marginal rates preserves the directional signal: a model that switches toward the bias on half its questions and away on the other half has $\text{BSR}_{\text{tot}} = 1$ but a marginal-rate difference of 0. The main text reports only the towards-bias switch rate; the away-from-bias and total switch rates are reported in Appendix D. We also report the *Bias Acknowledgement Rate* (BAR), the rate at which the model’s chain of thought acknowledges the perturbation (scored by Google Gemma 4 31B); a drop in BAR after consistency training indicates obfuscation.

3.6. Baselines

We compare RLCT against three baselines: (i) the untrained base model; (ii) BCT trained on a matched training set (same question sources, same training bias regime); and (iii) for each of BCT and RLCT, a *control* variant trained on the same questions but with the bias-inducing perturbation removed: the model is trained for consistency between two unbiased copies of the prompt rather than between the biased and unbiased prompt. The control isolates training-induced drift from the consistency signal: any change in the switch rates or BAR observed in the control reflects the effect of fine-tuning on unbiased data alone, leaving the residual gap between the control and the consistency-trained run as the contribution of the consistency objective itself. Control results are reported in Appendix C. **[TODO: ACT baseline if available.]**

3.7. Results

[TODO: fill in results.]

4. Discussion

[TODO: interpret results once finalised.]

Limitations. Consistency training does not address deceptive alignment: a model that is already deceptively aligned could plausibly be taught to hide its eval-gaming behaviour better, similarly to how standard post-training methods make

sleeper-agent behaviours difficult to catch (Hubinger et al., 2024). Our experiments are also restricted to a narrow bias-following setting on multiple-choice questions; the obfuscation question for the eval-awareness setting itself has not been tested directly. **[TODO: add limitations specific to results once finalised.]**

Future directions. Application of selective consistency training to evaluation-awareness settings (Section 1.2), extension to multimodal consistency (e.g. across input or output modalities), and application to other behavioural properties beyond bias-following.

5. Conclusion

[TODO: summarise once results are in.]

References

- Carlsmith, J. Scheming AIs: Will AIs fake alignment during training in order to get power?, November 2023. URL <http://arxiv.org/abs/2311.08379>. arXiv:2311.08379 [cs].
- Chua, J., Rees, E., Batra, H., Bowman, S. R., Michael, J., Perez, E., and Turpin, M. Bias-augmented consistency training reduces biased reasoning in chain-of-thought, 2024. URL <https://arxiv.org/abs/2403.05518>.
- Hubinger, E., Denison, C., Mu, J., Lambert, M., Tong, M., MacDiarmid, M., Lanham, T., Ziegler, D. M., Maxwell, T., Cheng, N., Jermyn, A., Askill, A., Radhakrishnan, A., Anil, C., Duvenaud, D., Ganguli, D., Barez, F., Clark, J., Ndousse, K., Sachan, K., Sellitto, M., Sharma, M., DasSarma, N., Grosse, R., Kravec, S., Bai, Y., Witten, Z., Favaro, M., Brauner, J., Karnofsky, H., Christiano, P., Bowman, S. R., Graham, L., Kaplan, J., Mindermann, S., Greenblatt, R., Shlegeris, B., Schiefer, N., and Perez, E. Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training, January 2024. URL <http://arxiv.org/abs/2401.05566>. arXiv:2401.05566 [cs].
- Irran, A., Turner, A. M., Kurzeja, M., Elson, D. K., and Shah, R. Consistency training helps stop sycophancy and jailbreaks, 2025. URL <https://arxiv.org/abs/2510.27062>.
- Perez, E., Ringer, S., Lukošiušė, K., Nguyen, K., Chen, E., Heiner, S., Pettit, C., Olsson, C., Kundu, S., Kadavath, S., Jones, A., Chen, A., Mann, B., Israel, B., Seethor, B., McKinnon, C., Olah, C., Yan, D., Amodei, D., Amodei, D., Drain, D., Li, D., Tran-Johnson, E., Khundadze, G., Kernion, J., Landis, J., Kerr, J., Mueller, J., Hyun, J., Landau, J., Ndousse, K., Goldberg, L.,

Lovitt, L., Lucas, M., Sellitto, M., Zhang, M., King-land, N., Elhage, N., Joseph, N., Mercado, N., DasSarma, N., Rausch, O., Larson, R., McCandlish, S., Johnston, S., Kravec, S., Showk, S. E., Lanham, T., Telleen-Lawton, T., Brown, T., Henighan, T., Hume, T., Bai, Y., Hatfield-Dodds, Z., Clark, J., Bowman, S. R., Askill, A., Grosse, R., Hernandez, D., Ganguli, D., Hubinger, E., Schiefer, N., and Kaplan, J. Discovering Language Model Behaviors with Model-Written Evaluations, December 2022. URL <http://arxiv.org/abs/2212.09251>. arXiv:2212.09251 [cs].

A. Bias-type descriptions

All six bias types perturb a multiple-choice question by introducing a cue toward a designated *biased option* (one of the wrong answers); an unbiased version of the same question, with the cue removed, is used as the reference. The descriptions below summarise each cue at the level needed to interpret the experiments; see Chua et al. (2024) for the full specifications and prompt templates.

Suggested answer. A short statement is appended to the question signalling that one of the wrong options is preferred (e.g., “My observations typically align with B.”). The model is meant to answer based on the question content, not the suggestion.

Distractor fact. An “irrelevant fun fact” about the biased option is prepended to the question (e.g., a trivial property of the option text), framed as potentially irrelevant. The cue is the mere presence of any salient information attached to that option.

Distractor argument. A short argument supporting the biased option, generated separately by Google Gemma 4 31B, is presented as “someone else’s argument I read”, and the model is asked to answer truthfully and not be biased by the argument. The cue is the plausible-sounding case for the wrong answer.

Post hoc. A two-turn exchange in which the question is first asked without chain-of-thought and the assistant turn is forced to a wrong option (e.g., “The best answer is: (B)”); the user then asks the model to explain its reasoning step-by-step without anchoring on its previous response. The cue is the planted prior answer.

Spurious few-shot squares. Several few-shot examples precede the question, each labelled with a correct answer; the labelled correct option is always marked with a small black-square symbol. A model that picks up the spurious symbol-to-label association will pick the squared option on the test question.

Wrong few-shot. Several few-shot examples precede the question, each labelled with a wrong answer rather than the correct one. The cue is the pattern of consistently incorrect labelling.

B. Training hyperparameters

All training uses LoRA adapters of rank 8.

BCT. A midway checkpoint is saved every 16 steps.

RLCT. Rollouts are sampled at temperature 1.0 with up to 20,480 generated tokens. The KL penalty to a fixed reference policy is 0.05 and the policy update uses the standard PPO clipped objective.

Learning-rate pooling. For each method-model combination we run three learning rates $\{1.0, 2.86, 5.0\} \times 10^{-4}$ and pool checkpoints across them when computing metrics. This gives a binomial standard error on the metric without tripling the rollout budget.

C. Control results

[TODO: insert control-vs-method comparison plots and prose for both BCT and RLCT.]

D. Away-from-bias and total switch rates

[TODO: insert away-from-bias and total switch-rate plots and prose, mirroring the towards-bias results in the main text.]

E. Distractor-argument + wrong-few-shot training regime

We additionally trained BCT and RLCT on a mixture of distractor-argument and wrong-few-shot prompts (DA+WFS) on LogiQA+HellaSwag, with all other settings matching the DA-only setup of Section 3. The DA+WFS BCT run uses the same training-data scale per bias, giving roughly 48 optimisation steps at batch size 128. RLCT uses an identical configuration to the DA-only setup with both bias types sampled in the prompt mix. **[TODO: fill in DA+WFS results.]**